

Final_DataViz_RMarkdown_F1

2026-05-12

```
# Load packages
library(tidyverse)
```

```
## — Attaching core tidyverse packages — tidyverse 2.0.0 —
## ✓ dplyr      1.2.1      ✓ readr      2.2.0
## ✓ forcats    1.0.1      ✓ stringr    1.6.0
## ✓ ggplot2    4.0.2      ✓ tibble     3.3.1
## ✓ lubridate  1.9.5      ✓ tidyr      1.3.2
## ✓ purrr      1.2.2
## — Conflicts — tidyverse_conflicts() —
## ✗ dplyr::filter() masks stats::filter()
## ✗ dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(readxl)
library(lme4)
```

```
## Loading required package: Matrix
##
## Attaching package: 'Matrix'
##
## The following objects are masked from 'package:tidyr':
##
##     expand, pack, unpack
```

```
library(lmerTest)
```

```
##
## Attaching package: 'lmerTest'
##
## The following object is masked from 'package:lme4':
##
##     lmer
##
## The following object is masked from 'package:stats':
##
##     step
```

```
library(broom)
library(scales)
```

```
##
## Attaching package: 'scales'
##
## The following object is masked from 'package:purrr':
##
##     discard
##
## The following object is masked from 'package:readr':
##
##     col_factor
```

```
#Loading in the data
race_results <- read_excel("C:/Users/Saman/Downloads/f1_race_results_2020_2025.xlsx",sheet = "Race_Results")

weather_data <- read_excel("C:/Users/Saman/Downloads/f1_race_results_2020_2025.xlsx",sheet = "Weather_Per_Race")

names(race_results)
names(weather_data)

glimpse(race_results)
glimpse(weather_data)
```

```
#Data Cleaning - race_results
#Need to ensure that everything is being read correct as factors, numeric, or characters and make sure you don't get any weirdness with the finishes and the places they finished in
race_results <- race_results %>%
  mutate(Year = as.factor(Year),
         Points = as.numeric(Points),
         Win = ifelse(Finish_Position == "1", 1, 0),
         FastestLap = ifelse(Fastest_Lap_Rank == "1",1,0),
         FinishedRace = ifelse(str_detect(Status, "Finished|\\+1 Lap|\\+2 Laps"),1,0),
         DNF = ifelse(FinishedRace == 1, 0, 1)
  )
```

```
#Data Cleaning - waether_data
#Need to ensure that everything is being read correct as factors, numeric, or characters and make sure you don't get any weirdness with the wind, no wind
weather_data <- weather_data %>%
  mutate(Year = as.factor(Year),
         Weather_Wind_NoWind = factor(
           Weather_Wind_NoWind,levels = c("No Wind","Low Wind","High Wind")
         )
  )
```

```
#Need to merge the weather data with the race results to make it easier to work with
combined_data <- race_results %>%
  left_join(weather_data,by = c("Year","Grand_Prix")
)
```

```
## Warning in left_join(., weather_data, by = c("Year", "Grand_Prix")): Detected an unexpected m
any-to-many relationship between `x` and `y`.
## i Row 781 of `x` matches multiple rows in `y`.
## i Row 1 of `y` matches multiple rows in `x`.
## i If a many-to-many relationship is expected, set `relationship =
## "many-to-many"` to silence this warning.
```

```
#Data Checking
glimpse(combined_data)
```

```
## Rows: 37,502
## Columns: 31
## $ Year                <fct> 2020, 2020, 2020, 2020, 2020, 2020, 2020, 2020, 2020, ...
## $ Round.x             <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...
## $ Grand_Prix           <chr> "Austrian Grand Prix", "Austrian Grand Prix", "A...
## $ Race_Date            <dtm> 2020-07-05, 2020-07-05, 2020-07-05, 2020-07-05,...
## $ Driver_Code.x        <chr> "BOT", "LEC", "NOR", "HAM", "SAI", "PER", "GAS",...
## $ Driver.x             <chr> "Valtteri Bottas", "Charles Leclerc", "Lando Nor...
## $ Driver_Nationality   <chr> "Finnish", "Monegasque", "British", "British", "...
## $ Constructor.x        <chr> "Mercedes", "Ferrari", "McLaren", "Mercedes", "M...
## $ Grid.x               <dbl> 1, 7, 3, 5, 8, 6, 12, 14, 18, 11, 20, 13, 4, 19,...
## $ Finish_Position.x    <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, NA, N...
## $ Points.x             <dbl> 25, 18, 16, 12, 10, 8, 6, 4, 2, 1, 0, 0, 0, 0, 0...
## $ Laps.x               <dbl> 71, 71, 71, 71, 71, 71, 71, 71, 71, 71, 71, 69, ...
## $ Fastest_Lap_Rank      <dbl> 2, 4, 1, 3, 5, 6, 11, 10, 9, 8, 16, 13, 7, 12, 1...
## $ Fastest_Lap_Time      <chr> "1:07.657", "1:07.901", "1:07.475", "1:07.712", ...
## $ Fastest_Lap_Speed     <dbl> 229.758, 228.933, 230.378, 229.572, 228.687, 227...
## $ Status.x             <chr> "Finished", "Finished", "Finished", "Finished", ...
## $ Weather_Wind_NoWind.x <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ...
## $ Win                  <dbl> 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, NA, NA, N...
## $ FastestLap            <dbl> 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
## $ FinishedRace         <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, ...
## $ DNF                  <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, ...
## $ Round.y              <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ...
## $ Driver_Code.y        <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ...
## $ Driver.y             <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ...
## $ Constructor.y        <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ...
## $ Grid.y               <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ...
## $ Finish_Position.y    <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ...
## $ Points.y             <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ...
## $ Laps.y               <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ...
## $ Status.y             <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ...
## $ Weather_Wind_NoWind.y <fct> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ...
```

```
summary(combined_data$Weather_Wind_NoWind)
```

```
## Warning: Unknown or uninitialised column: `Weather_Wind_NoWind`.
```

```
## Length Class Mode
##      0  NULL  NULL
```

```
tail(combined_data)
```

```
## # A tibble: 6 × 31
##   Year Round.x Grand_Prix      Race_Date      Driver_Code.x Driver.x
##   <fct>   <dbl> <chr>          <dtm>          <chr>      <chr>
## 1 2025      24 Abu Dhabi Grand Prix 2025-12-07 00:00:00 COL      Franco C...
## 2 2025      24 Abu Dhabi Grand Prix 2025-12-07 00:00:00 COL      Franco C...
## 3 2025      24 Abu Dhabi Grand Prix 2025-12-07 00:00:00 COL      Franco C...
## 4 2025      24 Abu Dhabi Grand Prix 2025-12-07 00:00:00 COL      Franco C...
## 5 2025      24 Abu Dhabi Grand Prix 2025-12-07 00:00:00 COL      Franco C...
## 6 2025      24 Abu Dhabi Grand Prix 2025-12-07 00:00:00 COL      Franco C...
## # i 25 more variables: Driver_Nationality <chr>, Constructor.x <chr>,
## #   Grid.x <dbl>, Finish_Position.x <dbl>, Points.x <dbl>, Laps.x <dbl>,
## #   Fastest_Lap_Rank <dbl>, Fastest_Lap_Time <chr>, Fastest_Lap_Speed <dbl>,
## #   Status.x <chr>, Weather_Wind_NoWind.x <lgl>, Win <dbl>, FastestLap <dbl>,
## #   FinishedRace <dbl>, DNF <dbl>, Round.y <dbl>, Driver_Code.y <chr>,
## #   Driver.y <chr>, Constructor.y <chr>, Grid.y <dbl>, Finish_Position.y <dbl>,
## #   Points.y <dbl>, Laps.y <dbl>, Status.y <chr>, Weather_Wind_NoWind.y <fct>
```

```
#Data Exploration
```

```
#should be the same as how much is in the excel
```

```
summary(race_results)
```

```
race_results %>%
  count(Year)
```

```
race_results %>%
  count(Weather_Wind_NoWind)
```

```
race_results %>%
  count(Driver, sort = TRUE)
```

```

#Driver Performances
driver_summary <- combined_data %>%
  group_by(Driver.x) %>%
  summarise(
    Races = n(),
    TotalPoints = sum(Points.x, na.rm = TRUE),
    MeanPoints = mean(Points.x, na.rm = TRUE),
    Wins = sum(Win, na.rm = TRUE),
    FastestLaps = sum(FastestLap, na.rm = TRUE),
    DNFs = sum(DNF, na.rm = TRUE),
    FinishRate = mean(FinishedRace, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(desc(TotalPoints))

driver_summary

```

#A mixed effects model was used to test whether race points differed among drivers while accounting for repeated observations across seasons. Driver was treated as a fixed effect and year was included as a random effect.

```

#Do drivers differ in points
points_model <- lmer(
  Points.x ~ Driver.x + (1 | Year),
  data = combined_data
)

summary(points_model)

```

```

##
## Correlation matrix not shown by default, as p = 39 > 12.
## Use print(x, correlation=TRUE) or
##      vcov(x)      if you need it

```

```

anova(points_model)

```

#Reliability Index ##The reliability index was calculated using standardized values of total points, mean points per race, wins, fastest laps, finish rate, and DNFs. Positive performance variables increased the reliability score, while DNFs decreased the score. Penalties were not included in this index so that the metric focused on fantasy-relevant race performance and consistency.

```

reliability_index <- combined_data %>%
  group_by(Driver.x) %>%
  summarise(
    Races = n(),
    TotalPoints = sum(Points.x, na.rm = TRUE),
    MeanPoints = mean(Points.x, na.rm = TRUE),
    Wins = sum(Win, na.rm = TRUE),
    FastestLaps = sum(FastestLap, na.rm = TRUE),
    DNFs = sum(DNF, na.rm = TRUE),
    FinishRate = mean(FinishedRace, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  mutate(
    ReliabilityScore =
      scale(TotalPoints)[,1] +
      scale(MeanPoints)[,1] +
      scale(Wins)[,1] +
      scale(FastestLaps)[,1] +
      scale(FinishRate)[,1] -
      scale(DNFs)[,1]
  ) %>%
  arrange(desc(ReliabilityScore))

reliability_index

```

```

#Creating a matrix for the visualization
reliability_index <- reliability_index %>%
  mutate(
    Driver = reorder(Driver.x, ReliabilityScore)
  )

```

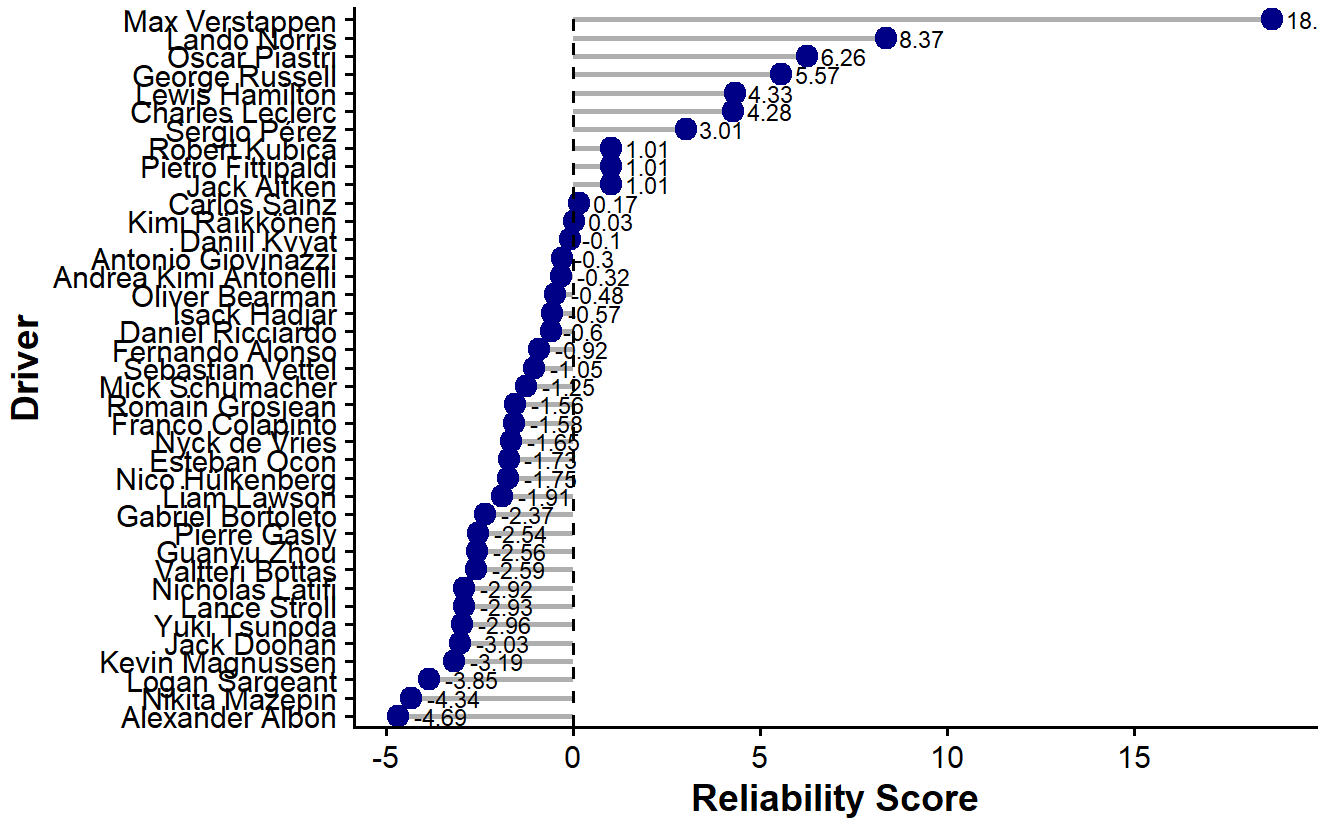
```

#Creating the visualization
reliability_plot <- ggplot(
  reliability_index,
  aes(x = ReliabilityScore, y = Driver)
) +
  geom_segment(
    aes(
      x = 0,
      xend = ReliabilityScore,
      y = Driver,
      yend = Driver
    ),
    linewidth = 1,
    color = "grey70"
  ) +
  geom_point(
    size = 3.5,
    color = "darkblue"
  ) +
  geom_text(
    aes(label = round(ReliabilityScore, 2)),
    hjust = -0.3,
    size = 3
  ) +
  geom_vline(
    xintercept = 0,
    linetype = "dashed",
    color = "black"
  ) +
  theme_classic(base_size = 14) +
  labs(
    title = "Formula 1 Driver Reliability Index, 2020-2025",
    subtitle = "Higher values indicate stronger fantasy-relevant reliability",
    x = "Reliability Score",
    y = "Driver"
  ) +
  theme(
    plot.title = element_text(face = "bold", hjust = 0.5),
    plot.subtitle = element_text(hjust = 0.5),
    axis.title = element_text(face = "bold")
  )
reliability_plot

```

Formula 1 Driver Reliability Index, 2020–2025

Higher values indicate stronger fantasy-relevant reliability



```
#saving the plot with specific size requirements
ggsave(
  "C:/Users/Saman/Downloads/reliability_plot.png",
  reliability_plot,
  width = 8,
  height = 16,
  dpi = 600
)
```

#Fastest Lap and Race Victory ##A logistic regression was used to test whether achieving fastest lap predicted whether a driver won the race. A chi-square test was also used to test whether fastest lap status and winning were statistically associated.

```
#Fastest Lap model
fastlap_win_model <- glm(
  Win ~ FastestLap,
  data = combined_data,
  family = "binomial"
)

summary(fastlap_win_model)
```



```
##
## Call:
## glm(formula = Win ~ FastestLap, family = "binomial", data = combined_data)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept) -3.14924    0.02825 -111.49  <2e-16 ***
## FastestLap   2.33636    0.05771  40.48  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##    Null deviance: 14481  on 33638  degrees of freedom
## Residual deviance: 13190  on 33637  degrees of freedom
## (3863 observations deleted due to missingness)
## AIC: 13194
##
## Number of Fisher Scoring iterations: 6
```

```
exp(coef(fastlap_win_model))
```

```
## (Intercept) FastestLap
## 0.0428848 10.3435199
```

#Need to combine the fastest lap statistics with the winning stats and then run a chisquare test on it

```
fastlap_table <- table(
  combined_data$FastestLap,
  combined_data$Win
)
```

```
fastlap_table
```

```
##
##           0      1
## 0 30477 1307
## 1 1285 570
```

```
fastlap_chisq <- chisq.test(fastlap_table)
fastlap_chisq
```

```
##
## Pearson's Chi-squared test with Yates' continuity correction
##
## data: fastlap_table
## X-squared = 2351.6, df = 1, p-value < 2.2e-16
```

```

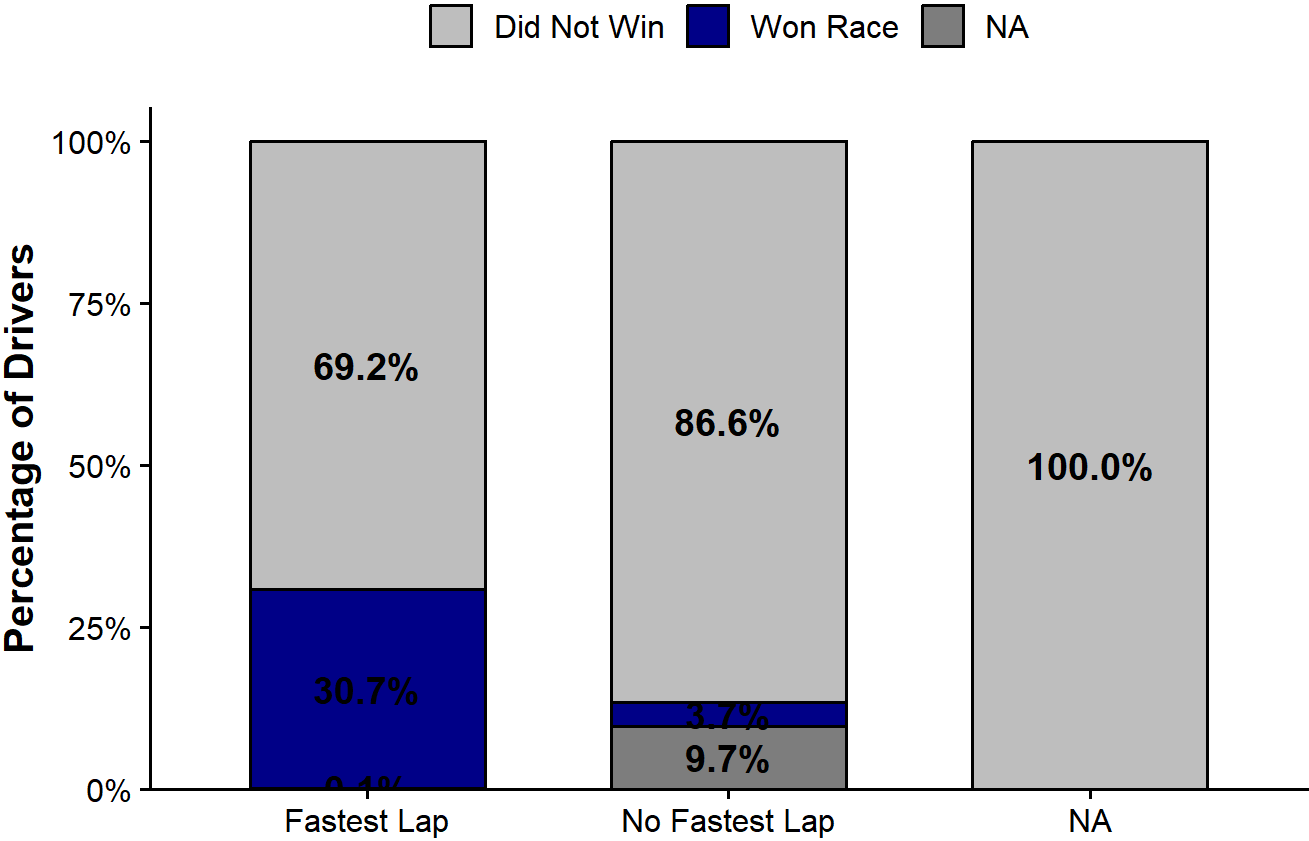
#making the matrix and the visualization for this
fastlap_plot_data <- combined_data %>%
  mutate(
    WinLabel = ifelse(Win == 1, "Won Race", "Did Not Win"),
    FastestLapLabel = ifelse(FastestLap == 1, "Fastest Lap", "No Fastest Lap")
  ) %>%
  group_by(FastestLapLabel, WinLabel) %>%
  summarise(n = n(), .groups = "drop") %>%
  group_by(FastestLapLabel) %>%
  mutate(Percent = n / sum(n))

#Visualization
fastlap_plot <- ggplot(
  fastlap_plot_data,
  aes(x = FastestLapLabel, y = Percent, fill = WinLabel)
) +
  geom_col(
    color = "black",
    width = 0.65
  ) +
  geom_text(
    aes(label = percent(Percent, accuracy = 0.1)),
    position = position_stack(vjust = 0.5),
    size = 5,
    fontface = "bold"
  ) +
  scale_y_continuous(
    labels = percent_format(),
    expand = expansion(mult = c(0, 0.05))
  ) +
  scale_fill_manual(
    values = c(
      "Won Race" = "darkblue",
      "Did Not Win" = "grey75"
    )
  ) +
  theme_classic(base_size = 15) +
  labs(
    title = "Relationship Between Fastest Lap and Race Victory",
    x = NULL,
    y = "Percentage of Drivers",
    fill = NULL
  ) +
  theme(
    plot.title = element_text(face = "bold", hjust = 0.5),
    axis.title = element_text(face = "bold"),
    legend.position = "top"
  )

fastlap_plot

```

Relationship Between Fastest Lap and Race Victory



```
#Saving the plot with specific settings
ggsave(
  "C:/Users/Saman/Downloads/fastest_lap_vs_winning.png",
  fastestlap_plot,
  width = 8,
  height = 6,
  dpi = 600
)
```

#Wind Condition and Driver Performance ##A mixed effects model was used to test whether driver performance differed across wind conditions. Driver, wind condition, and their interaction were included as fixed effects, while year was included as a random effect. The interaction term tested whether drivers responded differently to wind conditions.

```

#Need to filter the category because there are na's in the data and you don't want to include them
wind_summary <- combined_data %>%
  filter(!is.na(Weather_Wind_NoWind.y)) %>%
  group_by(Driver.x, Weather_Wind_NoWind.y) %>%
  summarise(
    Races = n(),
    MeanPoints = mean(Points.x, na.rm = TRUE),
    SE = sd(Points.x, na.rm = TRUE) / sqrt(n()),
    Wins = sum(Win, na.rm = TRUE),
    DNFs = sum(DNF, na.rm = TRUE),
    .groups = "drop"
  )

wind_summary

```

```

#Model creation and looking at the values
wind_model <- lmer(
  Points.x ~ Driver.x * Weather_Wind_NoWind.y + (1 | Year),
  data = combined_data
)

```

```

## fixed-effect model matrix is rank deficient so dropping 1 column / coefficient

```

```

summary(wind_model)

```

```

##
## Correlation matrix not shown by default, as p = 92 > 12.
## Use print(x, correlation=TRUE) or
##      vcov(x)          if you need it

```

```

anova(wind_model)

```

```

## Missing cells for: Driver.xJack Doohan:Weather_Wind_NoWind.yNo Wind.
## Interpret type III hypotheses with care.

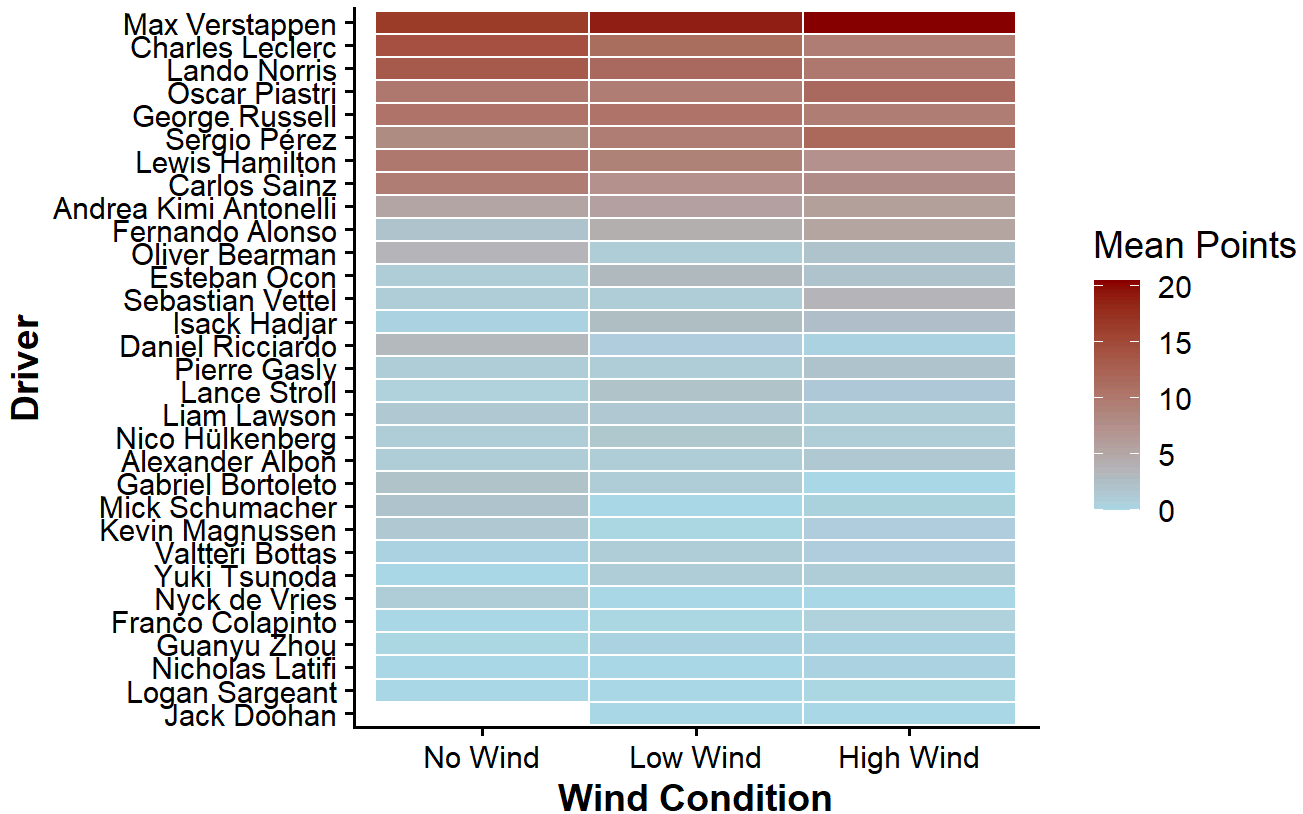
```

```
#Creating a visualization to look at how each driver does with respect to wind - heat map
wind_heatmap <- ggplot(
  wind_summary,
  aes(
    x = Weather_Wind_NoWind.y,
    y = reorder(Driver.x, MeanPoints),
    fill = MeanPoints
  )
) +
  geom_tile(
    color = "white",
    linewidth = 0.5
  ) +
  scale_fill_gradient(
    low = "lightblue",
    high = "darkred"
  ) +
  theme_classic(base_size = 14) +
  labs(
    title = "Mean Driver Points Across Wind Conditions",
    subtitle = "Higher values indicate stronger performance within each wind category",
    x = "Wind Condition",
    y = "Driver",
    fill = "Mean Points"
  ) +
  theme(
    plot.title = element_text(face = "bold", hjust = 0.5),
    plot.subtitle = element_text(hjust = 0.5),
    axis.title = element_text(face = "bold")
  )

wind_heatmap
```

Mean Driver Points Across Wind Conditions

Higher values indicate stronger performance within each wind category



```
#saving the image with different image settings
ggsave(
  "C:/Users/Saman/Downloads/wind_heatmap.png",
  wind_heatmap,
  width = 8,
  height = 12,
  dpi = 600
)
```

#Wind Adaptability Score ##Wind adaptability was calculated as the difference between mean points in no wind and high wind conditions. Smaller values indicate less performance decline under high wind conditions.

```
#Creating the matrix
wind_adaptability <- wind_summary %>%
  select(Driver.x, Weather_Wind_NoWind.y, MeanPoints) %>%
  pivot_wider(names_from = Weather_Wind_NoWind.y,
    values_from = MeanPoints) %>%
  mutate(WindDropoff = `No Wind` - `High Wind`) %>%
  arrange(WindDropoff)

wind_adaptability
```

```

#Visualization
wind_adaptability_plot <- ggplot(
  wind_adaptability,
  aes(
    x = WindDropoff,
    y = reorder(Driver.x, WindDropoff)
  )
) +
  geom_segment(
    aes(
      x = 0,
      xend = WindDropoff,
      y = Driver.x,
      yend = Driver.x
    ),
    color = "grey70",
    linewidth = 1
  ) +
  geom_point(
    size = 3.5,
    color = "darkred"
  ) +
  geom_vline(
    xintercept = 0,
    linetype = "dashed",
    color = "black"
  ) +
  theme_classic(base_size = 14) +
  labs(
    title = "Driver Wind Adaptability Score",
    subtitle = "Smaller values indicate less performance decline in high wind",
    x = "Point Drop-Off: No Wind - High Wind",
    y = "Driver"
  ) +
  theme(
    plot.title = element_text(face = "bold", hjust = 0.5),
    plot.subtitle = element_text(hjust = 0.5),
    axis.title = element_text(face = "bold")
  )

wind_adaptability_plot

```

```

## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_segment()`).

```

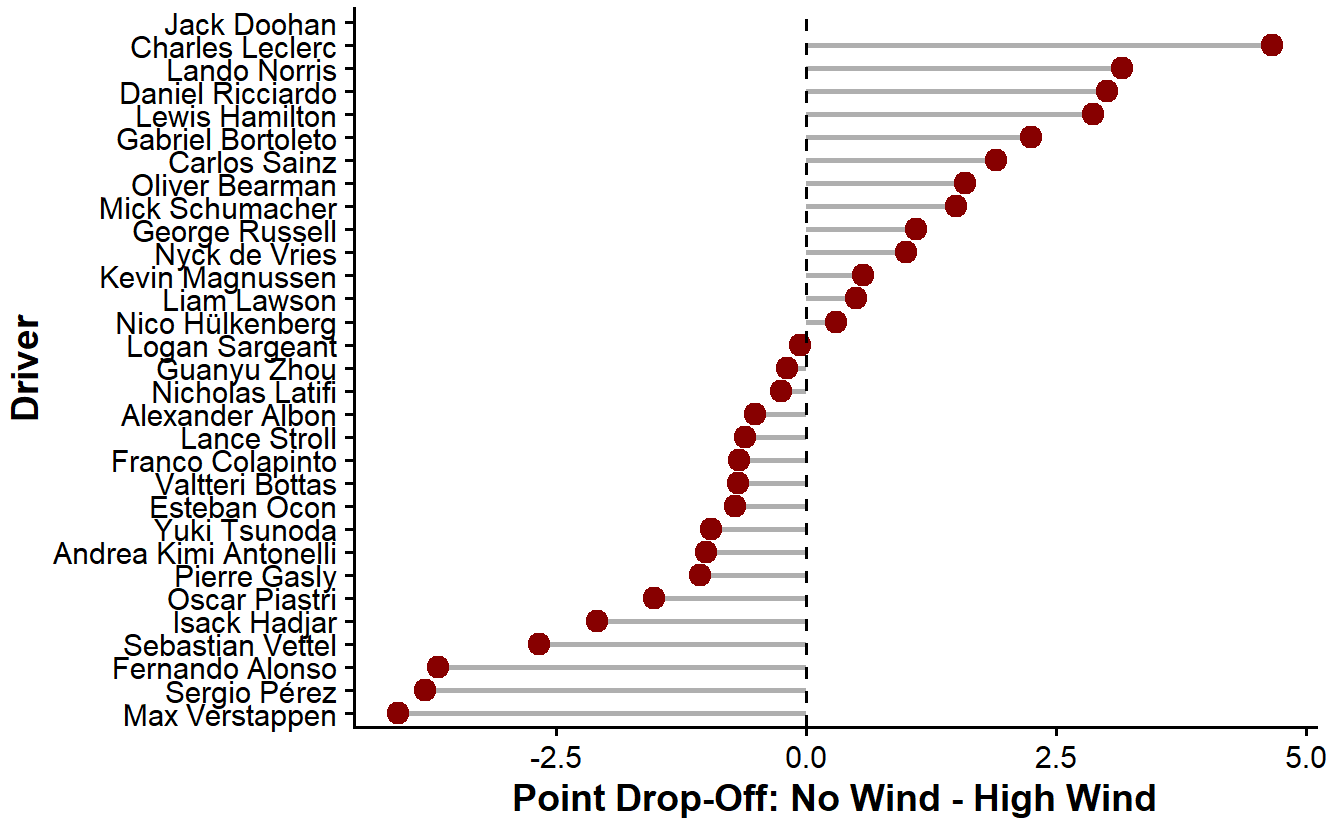
```

## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_point()`).

```

Driver Wind Adaptability Score

Smaller values indicate less performance decline in high wind



```
#Saving image with different settings
ggsave(
  "C:/Users/Saman/Downloads/wind_adaptability_plot.png",
  wind_adaptability_plot,
  width = 8,
  height = 12,
  dpi = 600
)
```

```
## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_segment()`).
```

```
## Warning: Removed 1 row containing missing values or values outside the scale range
## (`geom_point()`).
```

#Constructor Performance

#taking from the combined data set, you need to only look at the constructors and the points they got as well as if their drivers were winning and not finishing

```
constructor_summary <- combined_data %>%  
  group_by(Constructor.x) %>%  
  summarise(  
    Races = n(),  
    TotalPoints = sum(Points.x, na.rm = TRUE),  
    MeanPoints = mean(Points.x, na.rm = TRUE),  
    Wins = sum(Win, na.rm = TRUE),  
    DNFs = sum(DNF, na.rm = TRUE),  
    FinishRate = mean(FinishedRace, na.rm = TRUE),  
    .groups = "drop"  
  ) %>%  
  arrange(desc(TotalPoints))
```

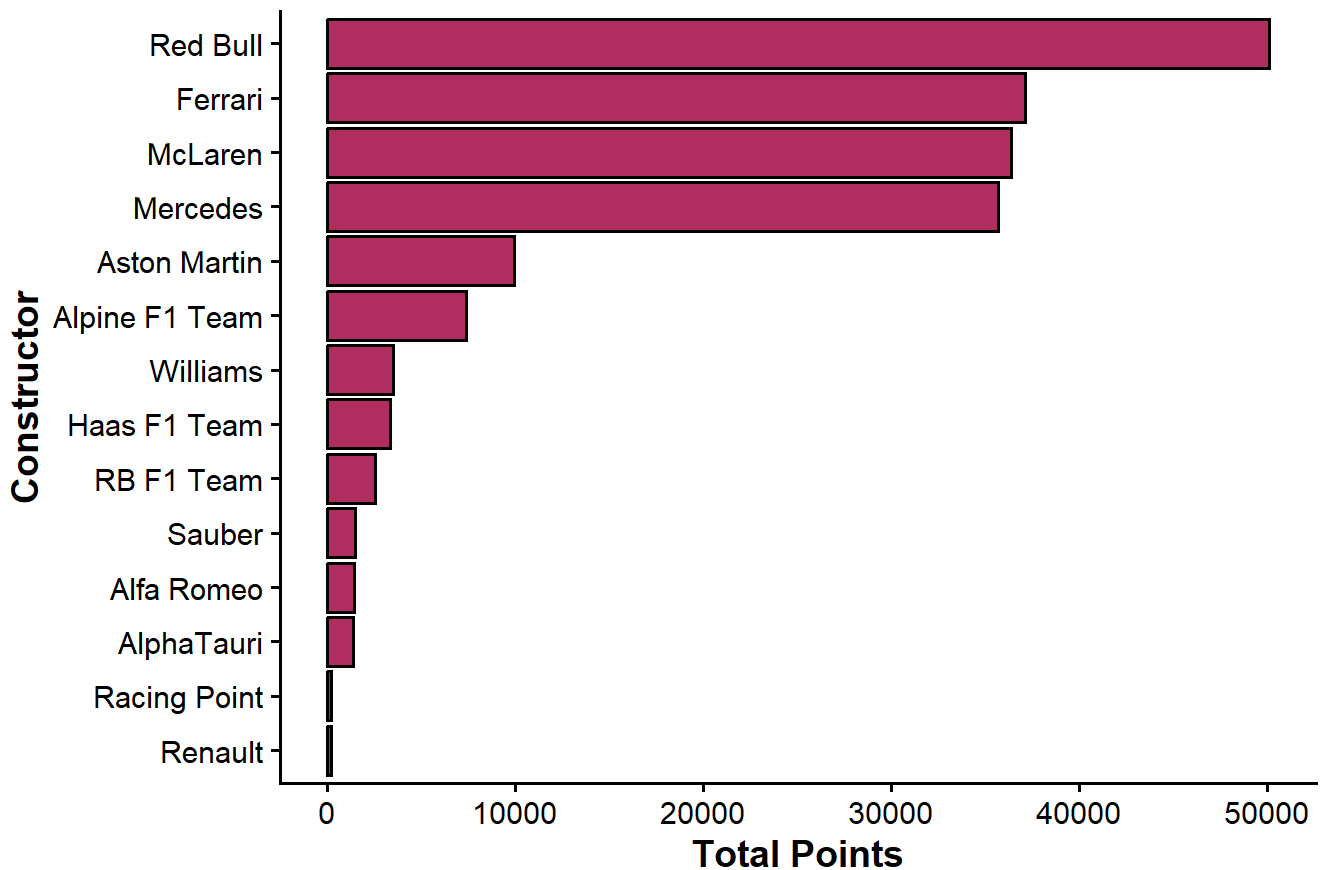
constructor_summary

#Visualization

```
constructor_plot <- ggplot(  
  constructor_summary,  
  aes(  
    x = TotalPoints,  
    y = reorder(Constructor.x, TotalPoints)  
  )  
) +  
  geom_col(  
    fill = "maroon",  
    color = "black"  
  ) +  
  theme_classic(base_size = 14) +  
  labs(  
    title = "Constructor Total Points, 2020-2025",  
    x = "Total Points",  
    y = "Constructor"  
  ) +  
  theme(  
    plot.title = element_text(face = "bold", hjust = 0.5),  
    axis.title = element_text(face = "bold")  
  )
```

constructor_plot

Constructor Total Points, 2020–2025



```
#Saving them with specific settings
ggsave(
  "C:/Users/Saman/Downloads/constructor_points.png",
  constructor_plot,
  width = 8,
  height = 6,
  dpi = 600
)
```

##Final Reflections **##1.** My rankings of my visualizations go in this order: formula one driver reliability index, Driver wind adaptability score, Constructor Total points, Mean driver points across weather condistions, and Relationship between fastest lap and race victory **##2.** The visualization that I was the most pleased with creating was the wind heatmap because it made it easy to compare driver performance across environmental categories. **##3.** Yes I did have some difficulties when getting the wind data. Race result data were available from open-source repositories, but weather and wind data were not available as one complete dataset. So, I had to go to blog posts and was only able to get the data from 2022-2025 unfortunately. **##4.** My project did evolve over the course of completing it. It began as a general Formula 1 fantasy performance analysis. It evolved to focus more specifically on reliability, fastest lap performance, and wind adaptability because these variables were measurable and directly relevant to fantasy team decisions. **##5.** Some challenges that I incountered while I was cleaning the data included cleaning race result columns, matching weather categories to races, and choosing statistical tests appropriate for repeated observations across seasons. I resolved these issues by keeping weather data in a separate sheet, merging the data by year and Grand Prix, and using mixed effects models to account for repeated yearly data. **##6.** If I had more time for this project, I would add qualifying performance, pit stop strategy, tire strategy, penalties, and actual race-day weather measurements instead of broader wind categories. **##7.** Yes. I am happy with the final project

because it combines sports analytics, statistical modeling, and data visualization in a way that directly answers fantasy Formula 1 questions and it was pretty fun to look into the data. ##8. Yes, you can use my project in future classes but please keep my name redacted. ##9. I would advise future students to choose a project with accessible data, clean their dataset early (which was a very rough part of mine), and focus on a few strong research questions instead of trying to analyze too many variables (which I tried to do initially).